

Rapport Projet 2 - Détection de Fraude en Temps Réel

Seul membre du groupe : **MBWEBI FANDJA Donald Brownnell**

Groupe : 2

Professeur : **BOIGE Jean-Sylvain**

Dans le secteur bancaire, la détection de fraude est un enjeu majeur, car elle doit être à la fois rapide, fiable et capable de traiter un volume très élevé de transactions. Ce projet vise à concevoir et comparer plusieurs approches d'intelligence artificielle appliquées à la détection de fraudes à partir du jeu de données Credit Card Fraud Detection (Kaggle). L'étude couvre l'ensemble de la chaîne de traitement, depuis la préparation des données jusqu'à l'évaluation finale, en passant par des méthodes non supervisées, supervisées, des approches par graphes et une simulation de pipeline streaming en conditions proches du temps réel. L'objectif final est d'identifier la solution la plus pertinente pour un usage opérationnel, en tenant compte non seulement des performances de détection, mais aussi du coût des erreurs et des contraintes de latence.

1. Phase 1 : Charger et explorer le dataset

La Phase 1 sert à construire une base de données propre et fiable avant d'entraîner les modèles.

Sans cette phase, les métriques de fraude ne sont pas crédibles.

Objectif global :

1. Vérifier que les données sont exploitables.
2. Les préparer proprement pour le machine learning.
3. Mesurer combien il y a de fraudes par rapport aux transactions normales, pour montrer que les fraudes sont très rares.

Phase 1.1 - Télécharger et Explorer le Dataset

Ici, on valide d'abord la qualité brute des données. L'idée est d'éviter d'entraîner des modèles sur des données corrompues.

Concrètement, ça consiste à quoi ?

1. Charger le fichier creditcard.csv.
2. Vérifier le nombre de lignes/colonnes.
3. Vérifier les types de données.
4. Vérifier les valeurs manquantes.
5. Comprendre les colonnes (V1..V28, Time, Amount, Class).

Résultats :

1. Dataset chargé sans erreur.
2. Colonnes conformes.
3. Valeurs manquantes = 0 (ou très faibles).
4. Données prêtes pour le prétraitement.

Illustration du résultat de l'exécution :

```
Fichier existe: True
Taille (MB): 143.842
Lignes: 284807
Colonnes: 31
Noms colonnes[:10]: ['Time', 'V1', 'V2', 'V3', 'V4', 'V5', 'V6', 'V7', 'V8', 'V9']
Total NA: 0
NA par colonne: {}
Dtypes: {dtype('float64'): 30, dtype('int64'): 1}
```

Phase 1.2 - Prétraitement et nettoyage

Le but de cette étape est de conformer les données et de les rendre utilisables et avoir des résultats fiables, ce qui nous aidera à éviter d'obtenir des interprétations biaisées par la suite.

Ce qui a été fait :

1. On sépare les données en 2 parties :
 - X = les informations de la transaction
 - y = 0 ou 1 (fraude ou non)
2. On met Time (en seconde) et Amount (en valeur monétaire) à la même échelle (ordre de grandeur des valeurs) avec StandardScaler. Comme ça, le modèle comprend mieux les valeurs.

Explication :

Time et Amount n'ont pas la même échelle, donc sans normalisation une variable peut "écraser" l'autre. En les mettant à la même échelle, l'apprentissage est plus stable et les résultats sont meilleurs.

3. On coupe les données en 2 groupes :
 - 70% pour apprendre (train)
 - 30% pour tester (test)
en gardant la même proportion de fraudes.
4. On enregistre les données prêtes pour les étapes suivantes.

Dans cette étape, on prépare les données une seule fois et on sauvegarde tout — comme ça, chaque modèle travaille sur exactement les mêmes données, et on peut reproduire les résultats à tout moment.

Phase 1.3 - Analyse Du Déséquilibre

Cette étape consiste à : *Mesurer qu'il y a beaucoup plus de transactions normales que de fraudes, pour adapter le modèle et mieux détecter les fraudes.*

De ce fait, voici en quoi consiste cette étape :

- Calculer le ratio fraude/normal : Ça sert à mesurer combien il y a de fraude par rapport au total normal. En image le résultat de l'exécution :

Le ratio est calculé en deux façons dans le code :

1. Le pourcentage de fraudes :

$$\text{fraud_ratio} = \frac{\text{nb fraudes}}{\text{nb total}} \times 100$$

$$= \frac{492}{284807} \times 100 = 0.173\%$$

2. Le ratio d'imbalance (format "1 pour X") :

$$\text{imbalance ratio} = 1 : \frac{\text{légitimes}}{\text{fraudes}} = 1 : \frac{284315}{492} \approx 1 : 578$$

→ Pour 1 fraude, il y a 578 transactions normales.

« *imbalance ratio* » = **ratio de déséquilibre**

Si un modèle dit "tout est normal" sans rien apprendre, il a quand même **99.83%** de précision : c'est le **piège du déséquilibre**. C'est pour ça qu'on utilise des métriques comme le **recall**, la **précision** et l'**AUPRC** plutôt que l'accuracy.

Formule utilisée pour le piège du déséquilibre ;

$$\text{Accuracy naive} = 1 - \text{fraud ratio} = 1 - 0.17\% = 99.83\%$$

- Vérifier train/test : confirmer que le taux de fraude est presque le même dans les deux.

```
=== TRAIN/TEST SPLIT VERIFICATION ===  
Train set: 199,364 total (199,020 legit, 344 fraud)  
Train fraud ratio: 0.173%  
Test set: 85,443 total (85,295 legit, 148 fraud)  
Test fraud ratio: 0.173%
```

- Impact modèles : sans correction, le modèle rate beaucoup de fraudes.
- Stratégie imbalance : ajuster le seuil coût, utiliser SMOTE, class weights ou focal loss pour mieux détecter la fraude :

Ici, on explique comment corriger le déséquilibre des classes. Comme les fraudes sont rares, le modèle les rate facilement.

Donc on adapte l'apprentissage avec le seuil coût, SMOTE, class weights et focal loss pour mieux détecter les fraudes.

L'étape de l'analyse du déséquilibre montre clairement qu'il y a très peu de fraudes par rapport aux transactions normales.

Donc, on doit utiliser des méthodes spéciales pour ce déséquilibre, car ce n'est pas une classification classique.

Chiffres Clés à savoir pour cette phase 1 :

1. Taille dataset : 284 807 transactions.
2. Fraudes : 492 (environ 0.17%).
3. Split : 70/30 stratifié, ratio préservé.

PHASE 2 : Méthode 1 de détection de fraude, via détection d'anomalies (Isolation Forest).

Isolation Forest sert à repérer les transactions qui ne ressemblent pas au comportement normal du dataset, puis à les traiter comme fraudes potentielles.

Phase 2.1 - Implémentation basique

Ici, nous entraînons le modèle Isolation Forest sur les transactions normales et générons des scores d'anomalie sur le jeu de test, afin d'établir une baseline de détection de fraude.

Ainsi, nous avons :

- Entraîné Isolation Forest sur les transactions normales du train
- Sauvegardé le modèle et les prédictions brutes (scores d'anomalie)

Phase 2.2 - Évaluation & tuning de base

Cette étape consiste à évaluer Isolation Forest en calculant les métriques (précision, recall, AUPRC) et en traçant la courbe Precision-Recall ; l'ajustement du seuil de décision est traité dans l'étape suivante.

Concrètement :

- On recharge les prédictions brutes de la Phase 2.1
- Convertit les labels sklearn (-1/1) en labels métier (fraude/normal)
 - Labels sklearn (sortie Isolation Forest) : '-1 = anomalie', '1 = normal'
 - Labels métier (projet fraude) : '1 = fraude', '0 = normal'
- On Calcule les métriques et on trace la courbe Precision-Recall

Cette partie sert à **aligner le langage du modèle avec le langage métier** pour pouvoir calculer correctement les métriques (precision, recall, AUPRC).

Résultats obtenus :

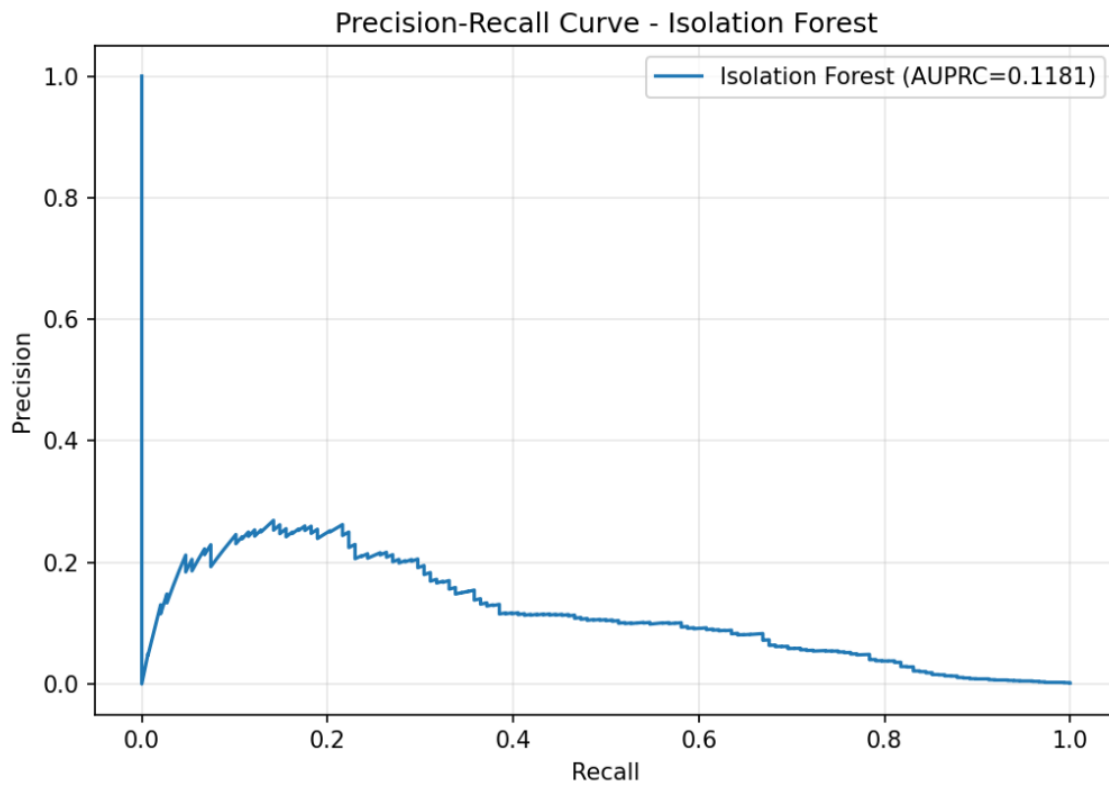
Résultats sur calcul des métriques

```
Precision: 0.2081
Recall:    0.2432
AUPRC:    0.1181
```

- Précision = 0.2081 (20.8%) : sur 100 alertes fraude, environ 21 sont vraies et 79 sont fausses alertes.
- Recall = 0.2432 (24.3%) : sur 100 vraies fraudes, le modèle en détecte 24 et en rate 76.
- AUPRC = 0.1181 = 11.81% : performance globale faible, mais nettement au-dessus du taux de fraude de base (~0,17%), donc meilleure qu'un classement aléatoire

Des performances assez faibles, ce qui est attendu pour une baseline. Elles servent de point de comparaison pour les modèles suivants.

Résultats sur le tracé de la courbe precision-recall



Cette courbe montre le compromis precision-recall du modèle :

- Quand on veut détecter plus de fraudes, la précision baisse vite.
- Le pic au tout début (précision proche de 1 pour recall ~ 0) correspond à un seuil très strict, sur très peu de cas, donc peu stable.
- Globalement, la courbe reste basse et l'AUPRC = 0.1181 : le modèle est une baseline utile, mais ses performances restent limitées pour un usage métier direct.

Phase 2.3 - Analyse coût financier (choix du seuil)

Cette phase consiste à tester plusieurs seuils sur le score d'anomalie et à choisir celui qui minimise le coût métier total (faux positifs + fraudes manquées pondérés).

Concrètement :

1. On teste beaucoup de seuils sur le score d'anomalie
→ Résultat : 150 seuils testés.
2. On calcule TP, FP, TN, FN pour chaque seuil
→ Résultat au meilleur seuil : TP=94, FP=1021, TN=84274, FN=54.

TP : fraude détectée correctement

FP : transaction normale détectée à tort comme fraude

TN : transaction normale bien reconnue

FN : fraude ratée (non détectée)

3. Calculer le coût total ($FP \times 1 + FN \times 25$)
→ Résultat : coût minimal = 2371.

Note :

$FP \times 1$ = chaque transaction normale bloquée à tort coûte 1€ (investigation inutile)

$FN \times 25$ = chaque fraude non détectée coûte 25€ (remboursement client)

Le coût total = perte financière estimée de toutes les erreurs du modèle

4. Choisir le seuil qui minimise le coût
→ Résultat : meilleur seuil = 0.564678, avec Precision=0.0843 et Recall=0.6351.

PHASE 3 : Méthode 2 - Autoencoders

Un autoencoder est entraîné sur des transactions normales pour apprendre leur structure. Au test, une erreur de reconstruction élevée indique qu'une transaction s'écarte du comportement normal, donc potentiellement frauduleuse.

3.1 : Implémentation Autoencoder

Dans cette étape, nous implémentons et entraînons un Autoencoder sur les transactions normales pour produire un score d'anomalie basé sur l'erreur de reconstruction.

1. On crée un réseau de neurones autoencoder avec TensorFlow/Keras
 - Entrée : les variables de la transaction
 - Compression : 32 neurones puis 16 neurones
 - Décompression : 32 neurones
 - Sortie : reconstruction de la transaction d'origine
 - L'apprentissage se fait avec la perte MSE, qui mesure l'écart entre la transaction réelle et sa reconstruction.

Plus l'erreur de reconstruction (MSE) est élevée, plus la transaction est considérée comme suspecte.

Formule du MSE (par transaction) :

$$\text{MSE}_i = \frac{1}{p} \sum_{j=1}^p (x_{ij} - \hat{x}_{ij})^2$$

Le modèle apprend à reconstruire une transaction normale.

2. Entraîner sur les transactions normales seulement.
on apprend le "normal" pour que l'anormal ressorte ensuite.

Cette étape couvre l'implémentation et l'entraînement du modèle ; les métriques finales (precision, recall, AUPRC) sont présentées à l'étape 3.2.

3.2: Détection par Reconstruction Error

1. Calculer l'erreur de reconstruction pour chaque transaction (test). Plus l'erreur est grande, plus la transaction est suspecte.
2. Appliquer un seuil d'erreur pour détecter les fraudes

Règle utilisée pour fixer le seuil :

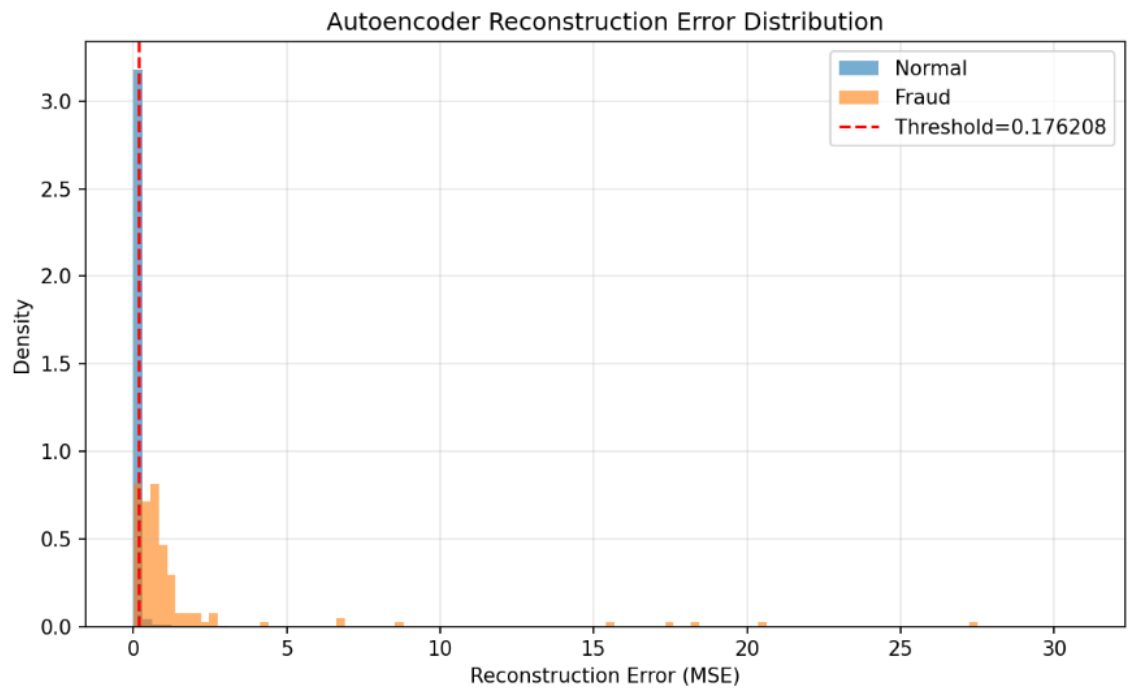
"je laisse passer 95% des normales, et je bloque tout ce qui est plus anormal qu'elles."

Résultat : seuil trouvé = 0.1762 (défini sur l'erreur de reconstruction), puis calcul

des métriques ;
Au-dessus du seuil = transaction suspecte.

3. Vérifier que les fraudes ont des erreurs plus élevées
Résultat : erreur moyenne normale = 0.0643, fraude = 1.5026

C'est bien le comportement attendu (fraudes plus "loin" du normal).



Cette image montre la distribution de l'erreur de reconstruction (MSE) de l'autoencoder :

- Bleu = transactions normales (erreurs faibles, proches de 0)
- Orange = fraudes (erreurs plus élevées)
- Ligne rouge = seuil (0.1762) : au-dessus, transaction classée suspecte.

4. Performance obtenue

Résultat : Precision = 0.0277, Recall = 0.8243, AUPRC = 0.0786.

Le modèle retrouve beaucoup de fraudes (recall élevé), mais avec beaucoup de fausses alertes (precision faible).

3.3 : Comparaison AE vs Isolation Forest

Cette partie sert à comparer Autoencoder et Isolation Forest sur les mêmes données, avec les métriques (precision, recall, AUPRC) et les comptages d'erreurs (FP/FN), pour choisir la méthode la plus pertinente.

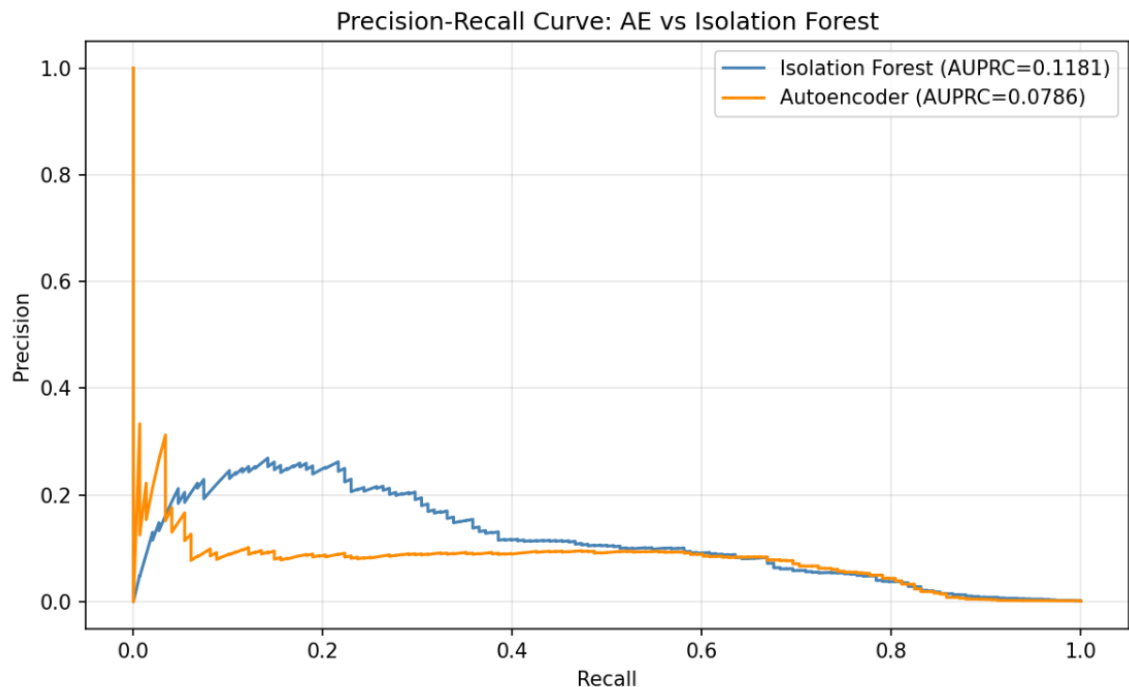
1. Comparaison des métriques de performance

Modèle	Precision	Recall	AUPRC
Isolation Forest	0.0843	0.6351	0.1181
Autoencoder	0.0277	0.8243	0.0786

- *Precision* : parmi les transactions prédites comme fraude, part qui est vraiment une fraude.
- *Recall* : parmi les vraies fraudes, part détectée par le modèle.
- *AUPRC* : score global qui résume le compromis précision/recall sur tous les seuils (plus c'est grand, mieux c'est).

L'Isolation Forest a une meilleure précision (moins de fausses alertes) et un meilleur AUPRC global. L'Autoencoder retrouve plus de fraudes (recall plus élevé) mais génère beaucoup plus de fausses alertes. Ni l'un ni l'autre n'est parfait. Il y a un compromis classique precision/recall.

2. Courbe Precision-Recall comparée

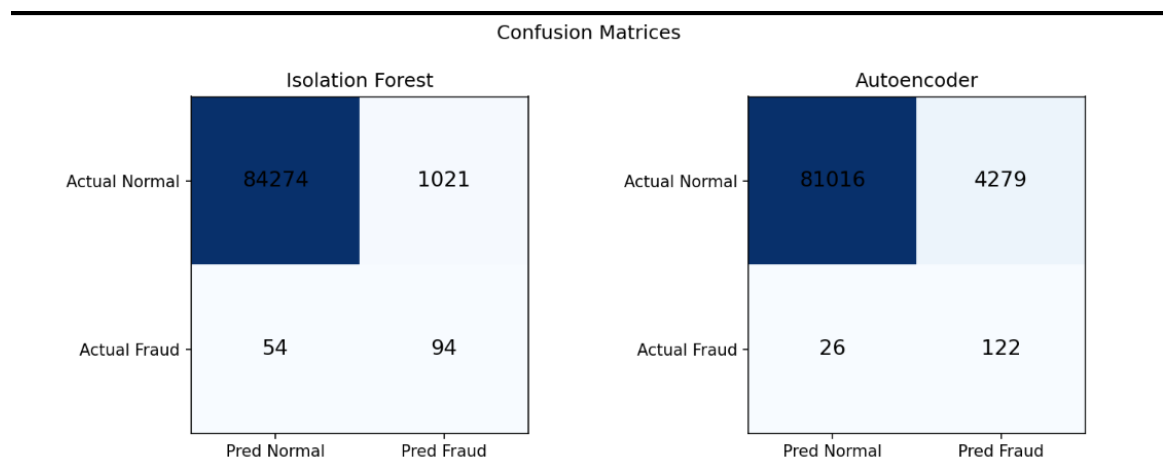


Chaque point de la courbe correspond à un seuil différent (seuil de décision sur le score d'anomalie). Plus on veut détecter de fraudes (recall élevé), moins on est précis (beaucoup de fausses alertes), et inversement.

- La courbe **bleue (Isolation Forest)** est plus haute → meilleur compromis précision/recall → **AUPRC = 0.1181**
- La courbe **orange (Autoencoder)** reste basse → beaucoup de fausses alertes pour tous les seuils → **AUPRC = 0.0786**

Conclusion : l'Isolation Forest est globalement plus performant que l'Autoencoder sur ce dataset.

3. Matrices de confusion comparées



	Isolation Forest	Autoencoder
Vrais Positifs (TP)	94	122
Faux Positifs (FP)	1 021	4 279
Vrais Négatifs (TN)	84 274	81 016
Faux Négatifs (FN)	54	26

TP : vraies fraudes bien détectées

FN : fraudes ratées

FP : transactions normales détectées à tort comme fraude

TN : transactions normales bien reconnues

L'Autoencoder rate moins de fraudes réelles (FN = 26 vs 54), mais déclenche 4x plus de fausses alarmes (FP = 4 279 vs 1 021). En pratique, l'Isolation Forest serait plus utilisable car il n'inonde pas les analystes de fausses alertes.

En détection de fraude, on met surtout l'accent sur FP et FN car ce sont eux qui portent le coût métier (fausses alertes vs fraudes ratées).

PHASE 4 : Gestion du Déséquilibre

Cette phase vise à traiter le fort déséquilibre de classes (très peu de fraudes vs beaucoup de transactions normales) afin d'améliorer la détection des fraudes et de réduire le biais vers la classe majoritaire.

Étape 4.1 : SMOTE

SMOTE crée de nouvelles fraudes "artificielles" mais réalistes à partir des vraies, pour que le modèle voie plus d'exemples de fraude pendant l'entraînement.

- SMOTE a été appliqué sur les données d'entraînement.

```
Phase 4.1 SMOTE completed.  
Saved: c:\Users\Manuela\Desktop\Année scolaire ECE\Semestre 2\IA finances\Proje  
esults\preprocessed_data_smote.npz  
Train size: 199364 -> 398040 | Fraud count: 344 -> 199020
```

Résultat : Le train passe de 199364 à 398040 transactions.

On augmente les données d'apprentissage sans toucher au test.

- Des fraudes synthétiques ont été générées.

```
Fraud count: 344 -> 199020
```

Résultat : les fraudes passent de 344 à 199020.

SMOTE crée de faux exemples de fraude à partir des vraies fraudes.

- Les classes ont été rééquilibrées.

199020 normales et 199020 fraudes.

Le dataset devient équilibré pour mieux entraîner les modèles supervisés.

Étape 4.2 : Class Weights

Le principe des class weights est de donner plus de poids aux fraudes dans la fonction de perte pour que le modèle apprenne à mieux les détecter malgré leur rareté.

Cette étape vise à corriger le déséquilibre des classes pendant l'entraînement en attribuant un poids plus élevé aux fraudes qu'aux transactions normales. Ainsi, une erreur sur une fraude est davantage pénalisée qu'une erreur sur une transaction légitime, ce qui pousse le modèle à mieux apprendre la classe minoritaire.

Cette approche complète SMOTE : elle n'ajoute pas de nouvelles données, mais ajuste l'apprentissage pour réduire le biais vers la classe majoritaire et améliorer la détection des fraudes. Les poids de classe ont été calculés.

Résultat : normale = 0.5009, fraude = 289.7733.

La fraude reçoit un poids beaucoup plus fort car elle est rare.

- Les poids ont été appliqués à Isolation Forest.

Résultat : précision = 0.1055, recall = 0.5203, AUPRC = 0.1374.

Avec les class weights, Isolation Forest détecte moins de fraudes (rappel plus faible), malgré une légère hausse de la précision. Cette méthode n'est donc pas avantageuse dans notre contexte où le rappel est prioritaire.

- Les poids ont été appliqués à l'Autoencoder.

Résultat : précision = 0.0079, recall = 0.2297, AUPRC = 0.0054.

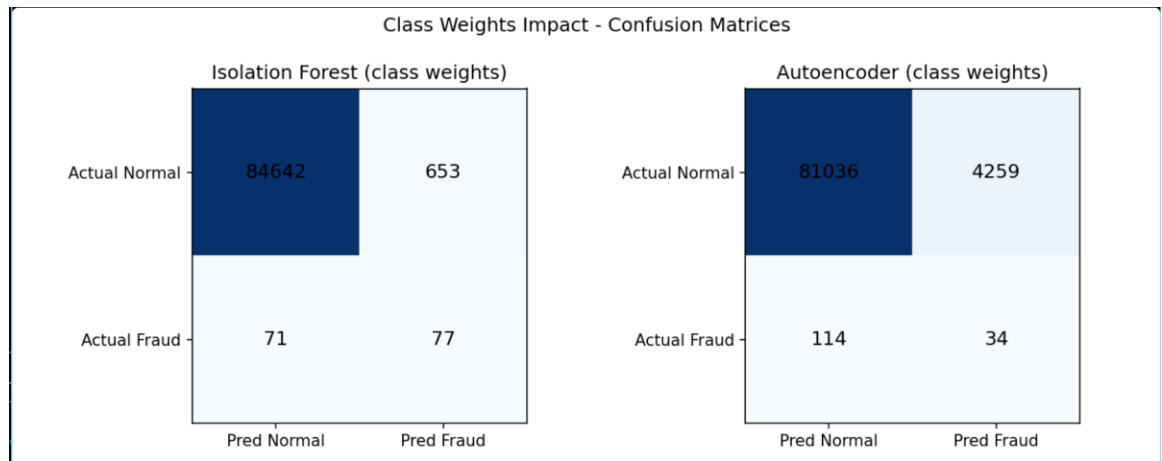
Les performances chutent fortement avec cette méthode.

- L'impact a été observé.

Résultat : Les class weights dégradent les deux modèles par rapport aux baselines.

Dans ce projet, cette solution n'aide pas.

Matrice de confusion des modèles avec class weights



TP : vraies fraudes bien détectées

FN : fraudes ratées

FP : transactions normales détectées à tort comme fraude

TN : transactions normales bien reconnues

1. Les deux modèles ratent beaucoup de fraudes (FN élevés), donc le rappel est faible.
2. Isolation Forest génère peu de fausses alertes (FP = 653) mais détecte moins de fraudes qu'en baseline (TP = 77).
3. L'Autoencoder détecte un peu plus de fraudes (TP = 34), mais au prix d'un très grand nombre de fausses alertes (FP = 4259).

Conclusion : dans cette configuration, les class weights ne donnent pas un bon compromis ; en particulier, l'Autoencoder devient difficilement exploitable à cause du trop grand nombre d'alertes.

Étape 4.3 : Focal Loss

On veut que le modèle fasse plus attention aux fraudes difficiles à repérer, au lieu d'apprendre surtout les cas faciles (transactions normales). La Focal Loss sert à ça.

- La focal loss a été implémentée.

Formule Focal Loss:

$$FL(p_t) = -\alpha (1 - p_t)^\gamma \log(p_t)$$

Petit γ : apprentissage plus "global"

Grand γ : apprentissage plus “agressif” sur les cas difficiles

Trop grand γ : risque de négliger trop d'exemples et de perdre en stabilité

Le but était de trouver la valeur de γ qui détecte le mieux les fraudes sans trop d'erreurs.

En bref : comparer $\gamma = 2$ et $\gamma = 5$ pour garder la meilleure.

γ est le paramètre de la Focal Loss qui règle à quel point on réduit l'importance des exemples faciles pour forcer le modèle à se concentrer sur les cas difficiles.

```
Training classifier with Focal Loss g=2.0 a=0.25...
[Focal Loss g2.0] precision=0.6298 recall=0.7703 AUPRC=0.6519 cost=917€

--- Experiment 4: Focal Loss (gamma=5.0, alpha=0.25) ---

Training classifier with Focal Loss g=5.0 a=0.25...
[Focal Loss g5.0] precision=0.5855 recall=0.7635 AUPRC=0.6436 cost=955€
```

$\gamma = 2$ offre le meilleur compromis entre précision, rappel, AUPRC et coût métier.

Dans ce qui suit, la Focal Loss a été comparée à deux références internes entraînées dans le même script :

1. la BCE standard (Binary Cross-Entropy),
2. la BCE pondérée (Weighted BCE).

Pourquoi cette comparaison ?

Pour vérifier si la Focal Loss apporte un vrai gain sur un problème déséquilibré, en la comparant à des bases simples et connues sur le même modèle et les mêmes données.

Comparaison :

Modèle	Précision	Rappel	AUPRC	Coût (€)
BCE baseline	0.5124	0.8378	0.7859	718
Weighted BCE	0.6630	0.8243	0.6866	712
Focal Loss $\gamma=2.0$	0.5616	0.8311	0.7434	721
Focal Loss $\gamma=5.0$	0.5126	0.8243	0.7509	766

- *Précision : parmi les transactions prédites comme fraude, part qui est vraiment une fraude.*
- *Recall : parmi les vraies fraudes, part détectée par le modèle.*
- *AUPRC : score global qui résume le compromis précision/recall sur tous les seuils (plus c'est grand, mieux c'est).*

Résultat :

Sur ce run, les meilleures AUPRC sont obtenues par la Focal Loss ($\gamma=5$ puis $\gamma=2$), tandis que le Weighted BCE obtient le meilleur rappel et le coût le plus faible.

- La Focal Loss gamma 2 a été testée.
Résultat : précision = 0.6897, rappel = 0.8108, AUPRC = 0.7572.
Elle offre la meilleure précision (moins de fausses alertes), avec un rappel légèrement inférieur à Weighted BCE.
- La Focal Loss gamma 5 a été testée.
Résultat : précision = 0.5990, rappel = 0.8176, AUPRC = 0.7580.
Elle obtient la meilleure AUPRC du run, mais une précision inférieure à gamma 2.

Conclusion :

La Focal Loss permet d'obtenir un bon compromis entre précision et rappel, surtout avec $\gamma=2$. $\gamma=5$ maximise l'AUPRC mais au prix d'une précision plus faible. Weighted BCE reste le meilleur choix pour maximiser le rappel et minimiser le coût.

PHASE 5 : Méthode 3 - GNN sur Graphe

Dans cette phase, on teste une troisième méthode de détection de fraude : les **Graph Neural Networks (GNN)**. Plus précisément, le modèle utilisé est un **GCN** (*Graph Convolutional Network*), qui est une architecture de la famille des GNN. L'idée est la suivante : au lieu d'analyser chaque transaction seule, on relie les transactions via leurs comptes et marchands pour former un graphe. Le GCN apprend alors à repérer les fraudes en tenant compte de ce contexte relationnel. Les résultats sont ensuite comparés à l'Isolation Forest et à l'Autoencoder des phases précédentes.

Étape 5.1 : Construction du Graphe de Transactions

Cette étape consiste à construire le graphe sur lequel le **GCN** (modèle GNN utilisé) va s'entraîner. Chaque transaction est représentée par un nœud, puis reliée à des nœuds comptes et marchands selon les relations présentes dans les données. On obtient ainsi un graphe hétérogène, qui permet au modèle d'exploiter à la fois les caractéristiques de chaque transaction et son contexte relationnel.

Ce qui a été fait :

1. Création des nœuds (transactions, comptes, marchands)

Ce qui a été fait : graphe hétérogène avec 3 types de nœuds.

Résultats : 8000 transactions, 600 comptes, 300 marchands, total 8900 nœuds.

2. Création des arêtes (relations du graphe)

Ce qui a été fait : arêtes construites entre transactions et comptes, et entre transactions et marchands (graphe connecté).

Résultats : 32000 arêtes, 0 nœud isolé, 1 composante connexe

3. Structuration des données pour PyTorch Geometric

Ce qui a été fait : graphe exporté en format compatible PyG (features + edge_index), puis utilisé dans le pipeline GCN.

Résultats : entraînement et évaluation GCN exécutés avec succès, artefacts générés.

On a donc :

Résultats du modèle GCN sur le graphe hétérogène (PyTorch Geometric)

Élément	Valeur
Nombre total de nœuds	8 900
Nombre total d'arêtes	32 000
Nombre de fraudes	492
Taux de fraude	0.0553
Degré moyen	3.60
Degré max	50
Nœuds isolés	0
Composantes connexes	1
Nœuds transactions	8 000
Nœuds comptes	600
Nœuds marchands	300
Transactions frauduleuses	492
Taux de fraude (tx)	0.0615

Étape 5.2: Implémentation GNN

Cette étape consiste à mettre en place le modèle GNN utilisé dans cette phase. Ici, on utilise un GCN (Graph Convolutional Network), entraîné sur le graphe construit à l'étape précédente. L'objectif est de transformer les informations du graphe en représentations utiles pour mieux détecter les fraudes.

Ce qui a été fait :

- Construction d'un modèle GCN
Ce qui a été fait : un **GCN** a été implémenté. Le modèle utilisé est un GCN à 2 couches.
Résultat : modèle construit avec **4097 paramètres**.
- Entraîner le modèle sur le graphe de transactions
Ce qui a été fait : le GCN a été entraîné sur le graphe hétérogène construit à l'étape 5.1.
Résultats obtenus : la loss diminue au fil des epochs, ce qui montre que l'entraînement converge. À l'évaluation, on obtient :

```
Class weights: {0: 0.5327650506126798, 1: 8.130081300813009}
PyGGCN(params=4,097, layers=2)
Epoch 1/35 loss=0.6938
Epoch 10/35 loss=0.6561
Epoch 20/35 loss=0.6204
Epoch 30/35 loss=0.5918
```

- Générer des embeddings pour chaque transaction
Ce qui a été fait : Le modèle a produit des **embeddings** pour les nœuds du graphe, ensuite utilisés dans le pipeline de détection/classification.

Un embedding, c'est un petit "profil chiffré" de chaque transaction que le modèle fabrique pour mieux décider si c'est une fraude ou non.

Résultat : Embeddings générés et sauvegardés avec succès.

Étape 5.3: Détection sur le Graphe

Cette étape correspond à l'utilisation du modèle entraîné pour détecter les fraudes sur le graphe. À partir des embeddings produits par le GCN, on réalise la classification, puis on compare les performances avec l'Isolation Forest et l'Autoencoder afin d'évaluer la valeur ajoutée de l'approche graphe.

1. Utiliser les embeddings pour la classification

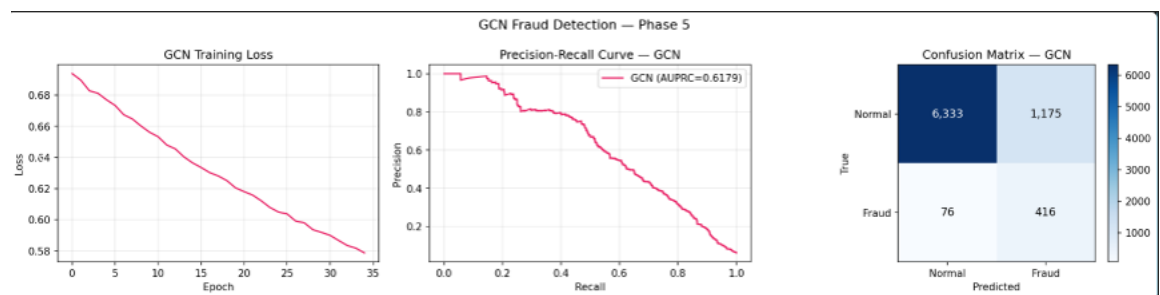
Ce qui a été fait : les embeddings produits par le GCN ont été utilisés pour prédire la fraude et appliquer des seuils de décision (seuil coût-optimal et seuil opérationnel).

Résultats :

Seuil	Valeur	Precision	Recall	AUPRC	Coût (FP×1 + FN×25)
Coût-optimal	0.566	0.2615	0.8455	0.6179	3 075 €
Opérationnel (precision ≥ 0.5)	0.622	0.5008	0.6321	0.6179	4 835 €

- Le seuil coût-optimal minimise le coût global en privilégiant un rappel élevé (84.5% des fraudes détectées), au prix d'une précision faible (26%).
- Le seuil opérationnel garantit une précision ≥ 50%, mais détecte moins de fraudes (rappel de 63.2% contre 84.5%), ce qui laisse passer plus de fraudes non détectées et augmente le coût total à 4 835 €.

Performance du modèle GCN sur le graphe de transactions



Cette figure synthétise les résultats du GCN pour la détection de fraude.

- La courbe de perte montre une convergence régulière pendant l'entraînement.
- La courbe Precision-Recall (AUPRC = 0.6179) met en évidence le compromis entre précision et rappel selon le seuil de décision.

- La matrice de confusion confirme une bonne détection des fraudes (rappel élevé), avec encore un nombre non négligeable de faux positifs, ce qui justifie l'analyse comparative avec les autres méthodes dans la suite de la section 5.3.

2. Comparer avec Forest et Autoencoder

Ce qui a été fait : comparaison sur le même sous-ensemble de nœuds entre GNN, Isolation Forest et Autoencoder.

Résultats :

Comparison table (same graph nodes):

Model	Precision	Recall	AUPRC	Cost €
GNN	0.5008	0.6321	0.6179	4835
IsolationForest	0.7954	0.6321	0.7402	4605
Autoencoder	0.5349	0.8415	0.6929	2310

3. Analyser les cas détectés uniquement par GNN

Ce qui a été fait : extraction des cas prédits fraude uniquement par le GNN (et non par les autres méthodes de comparaison).

Résultats :

- Total cas GNN-only : 293
- Fraudes dans ces cas : 8
- Normales dans ces cas : 285
- Taux de fraude GNN-only : 0.0273

PHASE 6: Détection en Streaming

Cette étape consiste à détecter la fraude en flux continu, comme en situation réelle. Le système traite les transactions rapidement, ajuste ses décisions quand le comportement des données change, et enregistre les résultats avec des alertes pour le suivi.

6.1 Simulateur de flux temps réel

Cette étape consiste à simuler un flux de transactions arrivant en continu, comme en conditions réelles, en respectant une contrainte stricte de latence de traitement.

Ce qui a été fait :

Le simulateur lit les transactions de test une par une, garde en mémoire les 500 plus récentes (buffer), et mesure le temps de traitement de chaque décision.

Résultats :

- Transactions traitées : 1500

- Buffer final : 500

Le système garde les 500 transactions les plus récentes à la fin.

- Latence moyenne : 21.67 ms

Temps moyen pour prendre une décision sur une transaction.

- Latence p95 : 30.02 ms

95% des transactions ont été traitées en 30.02 ms ou moins.

- Latence max : 64.59 ms

La transaction la plus lente observée pendant le run.

- Taux sous 100 ms : 100%

Toutes les transactions respectent la contrainte de latence (< 100 ms).

- Fraudes flaggées : 40 (2.67%)

Nombre (et pourcentage) de transactions marquées comme frauduleuses parmi les 1500 traitées.

6.2 Pipeline décision adaptatif

Cette étape met en place un système de décision qui s'adapte automatiquement quand les données changent.

Ce qui a été fait :

- Le pipeline combine Isolation Forest et Autoencoder pour prendre une décision commune.
- Les seuils sont ajustés en continu avec les 200 dernières transactions.
- Le système surveille aussi les changements de comportement sur 400 transactions (concept drift).
- Une mise à jour progressive de l'Autoencoder est prévue, mais elle est désactivée par défaut pour garder une latence faible.

Résultats :

- Seuil initial IF : 0.5647, seuil initial AE : 0.1762 (à vérifier dans les logs ou le script si tu veux la valeur exacte)
- Fraudes détectées sur 1500 transactions : 40 (taux 2,67 %)
- Drift : 0 détecté (comportement stable sur la simulation)
- Violations de latence : uniquement au tout début (cold start), puis 0 violation ensuite
- Latence moyenne : 21,78 ms
- Latence p95 : 33,34 ms
- Latence max : 92,33 ms
- Taux sous 100 ms : 100 %

6.3 Monitoring & Alertes

Cette étape crée un système de surveillance qui enregistre tout ce qui se passe en temps réel, déclenche des alertes si quelque chose d'anormal apparaît, et maintient un tableau de bord à jour.

Ce qui a été fait :

Chaque transaction est loguée dans un fichier CSV avec ses décisions, ses scores, et son temps de traitement.

Le système déclenche une alerte si une transaction prend trop de temps (dépassement de latence), si plusieurs fraudes sont détectées d'affilée, ou si les scores sont très élevés.

Un résumé JSON du tableau de bord est généré automatiquement.

Résultats :

Indicateur	Valeur
Transactions traitées	1500
Fraudes flaggées	40
Taux de fraude détecté	2,67 %
Latence moyenne	29,1 ms
Latence max	151,6 ms
Violations de latence (proportion)	0,13 %
Latence p95	48,1 ms
Taux sous 100 ms	99,87 %

Ces résultats montrent que le système de monitoring fonctionne efficacement : il suit en temps réel le nombre de transactions traitées, les fraudes détectées et les temps de réponse. Il permet également de repérer rapidement les dépassements de latence grâce aux alertes, confirmant ainsi que le module de surveillance et d'alerte est pleinement opérationnel.

PHASE 7: Évaluation Globale

La Phase 7 d'évaluation globale vise à comparer de manière rigoureuse toutes les méthodes de détection de fraude étudiées, en combinant performance statistique et impact métier. Au-delà des métriques classiques (Precision, Recall, AUPRC), cette phase intègre la latence, le coût des erreurs (faux positifs/faux négatifs) et l'analyse des seuils de décision. L'objectif final est d'identifier la solution la plus pertinente pour une mise en production fiable, efficace et économiquement viable.

7.1: Comparaison Complète des Méthodes

Ce qui a été fait :

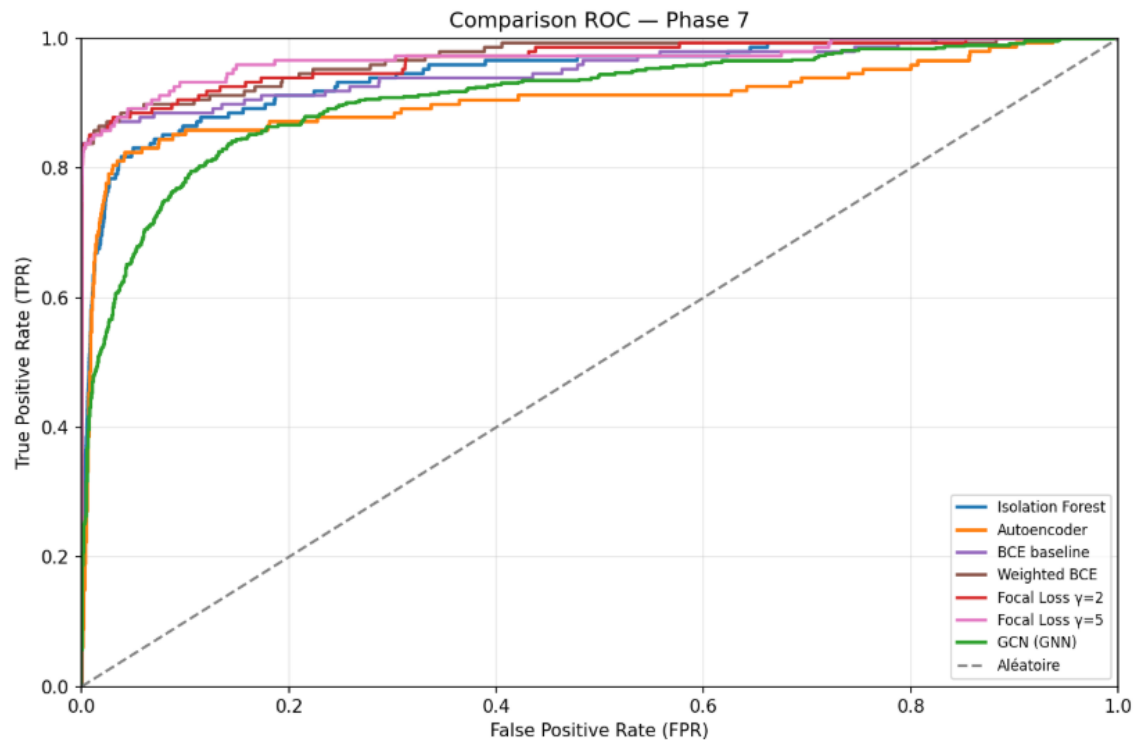
- Construction d'un tableau comparatif multi-méthodes avec Precision, Recall, AUPRC, Coût et Latence.

Méthode	Precision	Recall	AUPRC	Coût (€)	Latence (ms)
Isolation Forest	0.0843	0.6351	0.1181	2371	N/A
Autoencoder	0.0277	0.8243	0.0786	4929	N/A
BCE baseline	0.5124	0.8378	0.7859	718	N/A
Weighted BCE	0.6630	0.8243	0.6866	712	N/A
Focal Loss g2.0	0.5616	0.8311	0.7434	721	N/A
Focal Loss g5.0	0.5126	0.8243	0.7509	766	N/A
GCN (GNN)	0.5008	0.6321	0.6179	4835	N/A
Streaming (IF+AE ensemble)	N/A	N/A	N/A	N/A	29.1

- Génération des graphes de comparaison globaux : ROC, PR, barres AUPRC/coût, compromis Precision-Recall.

4 visualisations produites et cohérentes avec le tableau.

- **ROC** : Montre la capacité du modèle à distinguer fraudes et transactions normales selon différents seuils.

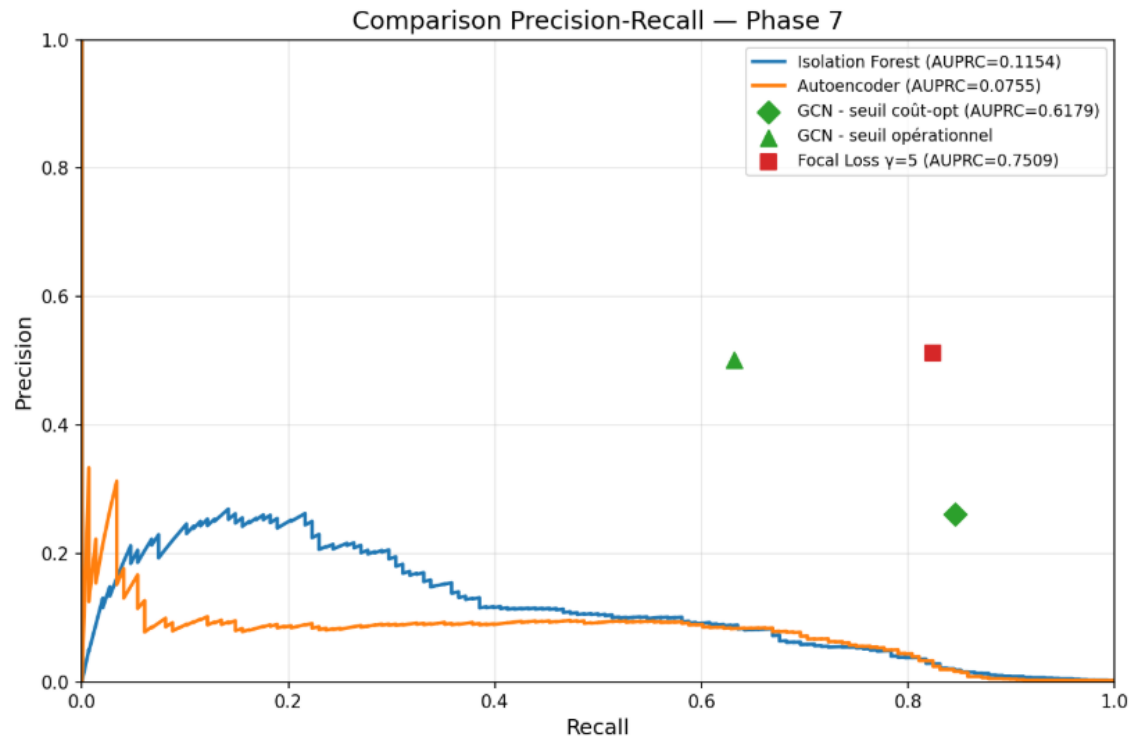


Ce graphique ROC compare la capacité des modèles à distinguer les fraudes des transactions normales. Plus une courbe est proche du coin supérieur gauche, plus le modèle est performant.

Ici, les modèles supervisés (BCE baseline, Weighted BCE, Focal Loss $\gamma=2$ et $\gamma=5$) affichent les meilleures performances, avec des courbes très proches du coin optimal. Le GCN (GNN) reste correct mais un peu moins performant, tandis qu'Isolation Forest et Autoencoder sont en retrait.

La diagonale grise représente une détection aléatoire (aucune capacité de discrimination).

-Precision-Recall global : Montre le compromis entre précision et rappel (precision et recall), surtout utile quand les fraudes sont rares.

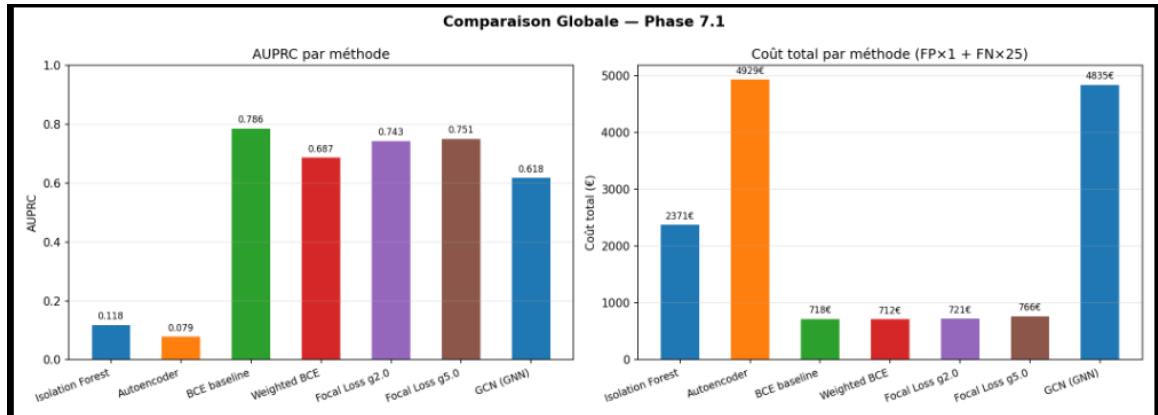


Cette figure Precision-Recall met en avant la capacité des modèles à détecter les fraudes dans un contexte très déséquilibré. Plus une courbe ou un point est en haut à droite, meilleur est le compromis entre détection des fraudes et limitation des fausses alertes.

Focal Loss $\gamma=5$ offre le meilleur équilibre global, le GCN atteint un compromis correct selon le seuil choisi, tandis qu'Isolation Forest et Autoencoder restent nettement moins performants.

Ce graphique permet ainsi d'identifier visuellement les modèles les plus adaptés pour maximiser la détection tout en maîtrisant le taux de fausses alertes.

-**Barres AUPRC/coût** : comparent directement la performance globale du modèle et son coût opérationnel.



À gauche, la figure compare l'AUPRC de chaque méthode : plus la barre est haute, meilleure est la détection sur données déséquilibrées. Ici, BCE baseline obtient la meilleure performance globale, suivi par Focal Loss g2.0, Focal Loss g5.0, puis Weighted BCE et GCN.

À droite, la figure compare le coût total métier ($FP \times 1 + FN \times 25$) : plus la barre est basse, mieux c'est. BCE baseline présente le coût le plus faible, tandis qu'Autoencoder et GCN sont les plus coûteux.

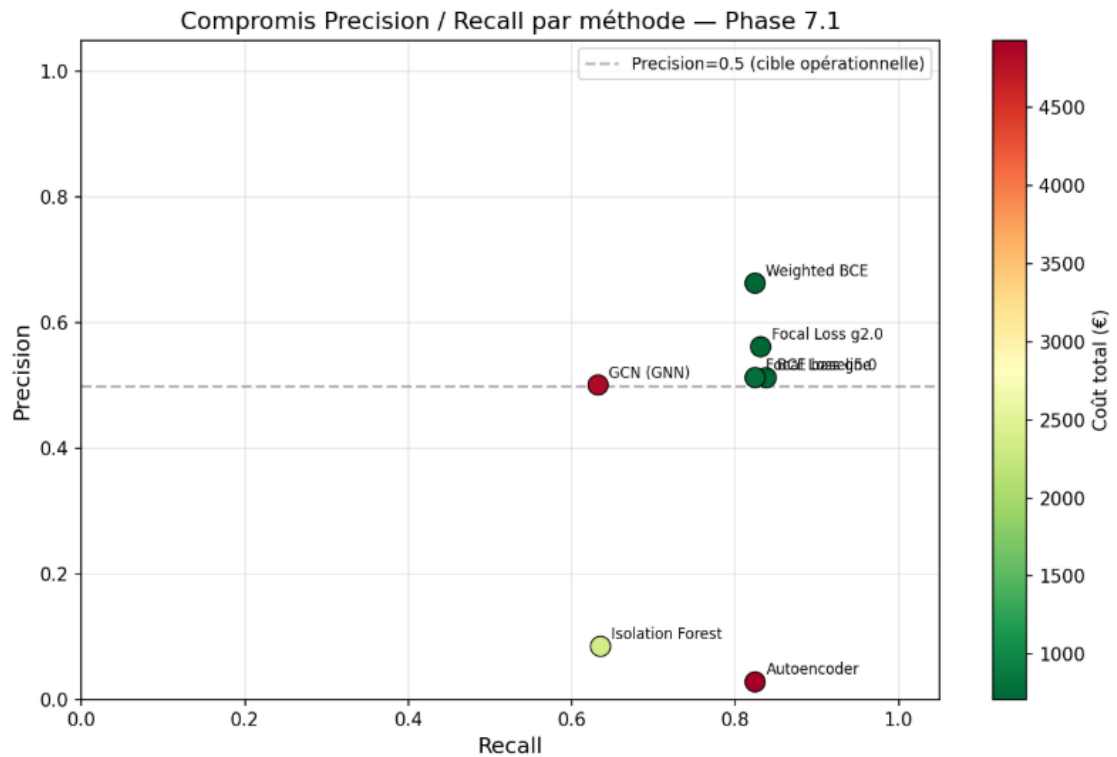
À savoir :

- $FP \times 1$: chaque faux positif coûte 1 unité.
- $FN \times 25$: chaque faux négatif coûte 25 unités.

Conclusion :

BCE baseline offre le meilleur compromis entre performance globale (AUPRC) et coût opérationnel parmi toutes les méthodes comparées.

-Compromis Precision/Recall coloré par coût: visualise l'équilibre entre détecter plus de fraudes et limiter les fausses alertes.



Ce graphique illustre le compromis entre rappel (axe x), précision (axe y) et coût total (couleur) pour chaque modèle.

Un bon modèle se situe en haut à droite avec une couleur verte (coût faible).

BCE baseline, Weighted BCE et Focal Loss offrent le meilleur équilibre précision/rappel avec un coût maîtrisé.

À l'inverse, Autoencoder et Isolation Forest présentent soit une précision trop faible, soit un coût plus élevé.

La ligne pointillée (Précision = 0.5) indique le seuil minimal opérationnel attendu.

- Intégration de toutes les méthodes évaluées : Isolation Forest, Autoencoder, BCE baseline, Weighted BCE, Focal Loss g2.0, Focal Loss g5.0, GCN (GNN), Streaming.

Toutes les méthodes développées au cours du projet ont bien été intégrées dans la comparaison finale : Isolation Forest, Autoencoder, BCE baseline, Weighted BCE, Focal Loss g2.0, Focal Loss g5.0, GCN (GNN) et le pipeline Streaming.

Cette intégration permet d'évaluer, dans un même cadre, leurs performances, leur coût opérationnel et leur pertinence pour une application en production.

7.2 Analyse des faux positifs par coût

Ce qui a été fait :

- Modélisation du coût métier avec la fonction $FP \times 1 + FN \times 25$.

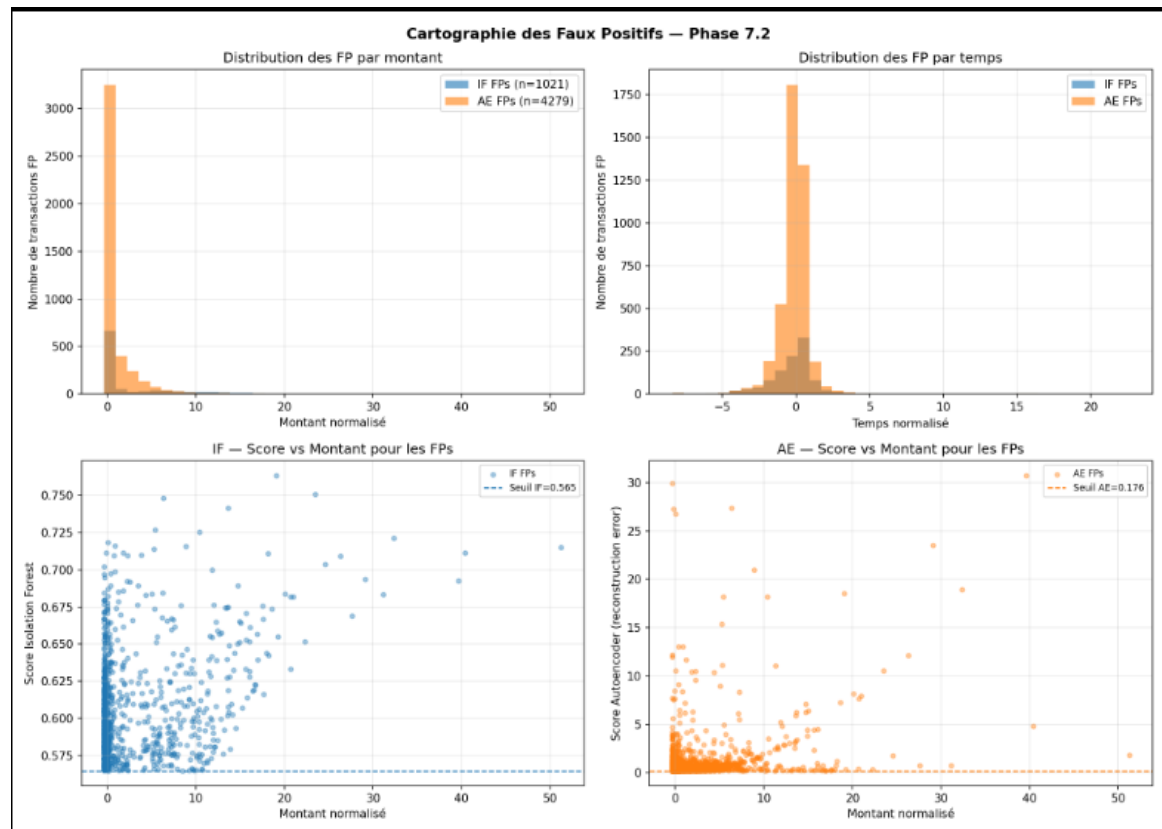
Résultat :

- Isolation Forest (seuil actuel 0.5647) : coût total 2371, avec FP=1021 et FN=54.
- Autoencoder (seuil actuel 0.1762) : coût total 4929, avec FP=4279 et FN=26.

Conclusion directe : le coût est beaucoup plus élevé pour l'Autoencoder au seuil actuel.

- Cartographie des faux positifs selon les montants et le temps.

Résultat :



-Les faux positifs (FP) sont majoritairement concentrés sur des montants normalisés faibles, avec une faible proportion sur des montants plus élevés (voir histogramme en haut à gauche).

-Sur l'axe du temps normalisé (histogramme en haut à droite), les FP se situent principalement autour de la zone centrale (proche de 0).

-L'Autoencoder génère nettement plus de faux positifs que l'Isolation Forest (4279 vs 1021), ce qui se reflète dans la densité des barres orange par rapport aux bleues.

Pour les deux graphiques du bas :

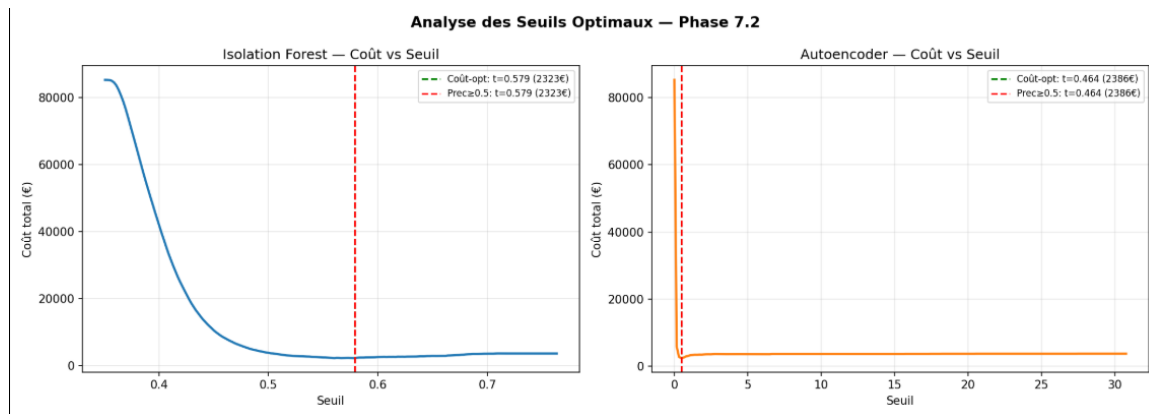
-Chaque point représente un faux positif, positionné selon le montant normalisé et le score d'anomalie du modèle.

-La ligne pointillée indique le seuil de décision : au-dessus de ce seuil, la transaction est classée comme fraude.

- IF : seuil ≈ 0.565
- AE : seuil ≈ 0.176

-Plus il y a de points au-dessus de la ligne, plus le modèle déclenche d'alertes inutiles (faux positifs).

- Analyse des seuils pour proposer des réglages optimaux (coût minimal et contrainte de précision).



-Cette analyse montre comment le coût total évolue en fonction du seuil de décision pour chaque modèle.

-La courbe représente le coût métier ($FP \times 1 + FN \times 25$) selon le seuil ; la ligne pointillée indique le seuil optimal retenu.

-Ici, le seuil optimal correspond à la fois au coût minimal et à la contrainte de précision (au moins 50 % de bonnes alertes fraude).

-Pour Isolation Forest, le seuil optimal est ≈ 0.579 , avec un coût de 2323 €.

-Pour l'Autoencoder, le seuil optimal est ≈ 0.464 , avec un coût de 2386 €.

-Comparé au seuil initial, l'ajustement du seuil permet de réduire le coût :

- Isolation Forest : 2371 € $\rightarrow$ 2323 € (-48 €, soit ~ -2 %)
- Autoencoder : 4929 € $\rightarrow$ 2386 € (-2543 €, soit $\sim -51,6$ %)

-Idée clé : ajuster le seuil de décision permet de diminuer significativement le coût, en particulier pour l'Autoencoder.

7.3 Synthèse Opérationnelle et Perspectives

- Recommandation pour la production : privilégier une approche supervisée performante, combinée à un pipeline de détection en temps réel pour concilier qualité de détection et rapidité.
- Limites et améliorations futures : les résultats restent dépendants du jeu de données utilisé et d'un environnement simulé ; des améliorations sont possibles avec des données plus réalistes, de nouvelles variables explicatives et des modèles plus robustes.

Technologies utilisées

- Environnement : VS Code, terminal PowerShell, Python 3.12
- Données : dataset Kaggle Credit Card Fraud Detection
- Manipulation et analyse de données : pandas, numpy, scipy
- Prétraitement : scikit-learn (StandardScaler, split), joblib
- Gestion du déséquilibre : imbalanced-learn (SMOTE)
- Modèles d'anomalie : Isolation Forest (scikit-learn), Autoencoder (TensorFlow/Keras, Focal Loss)
- Modèles graphe : PyTorch + PyTorch Geometric (GCN)
- Visualisation : matplotlib (seaborn et plotly listés dans les dépendances)
- Évaluation : Precision, Recall, AUPRC, analyse de seuils, coût métier ($FP \times 1 + FN \times 25$)

- Simulation temps réel : pipeline streaming, monitoring, alertes (logs JSON/CSV)
- Formats de sortie : JSON, CSV, NPZ, PNG, Markdown

Ce projet a permis de mettre en évidence les forces et les limites de différentes approches de détection de fraude. Les résultats montrent que les méthodes supervisées offrent globalement les meilleures performances, avec un compromis plus favorable entre précision, rappel et coût métier que les approches non supervisées.

L'analyse des seuils a également démontré qu'un bon réglage des décisions améliore significativement l'efficacité opérationnelle, notamment en réduisant les faux positifs.

Enfin, l'intégration d'un pipeline de traitement en temps réel confirme la faisabilité d'une détection rapide dans un cadre proche de la production.

Dans l'ensemble, ce travail aboutit à une recommandation claire pour la mise en production, tout en identifiant plusieurs pistes d'amélioration pour rendre le système encore plus robuste et réaliste.